



Instituto Tecnológico y de Estudios Superiores de Monterrey

Campus Estado de México

Inteligencia artificial avanzada para la ciencia de datos (Grupo 601)

Profesores: Jorge Adolfo Ramírez Uresti, Elisabetta Crescio, Julio G. Arriaga
Blumenkron, Israel Sánchez Miranda, Víctor Adrián Sosa Hernández, Miguel González
Mendoza, Alberto Michel Pérez Domínguez

Reto académico: *Spaceship Titanic*

Emilio Torres Paul | A01753010

Ana Paula Moreno Frías | A01751886

David Tinoco Romero | A01801491

Alejandro Vázquez De La Cruz | A01800791

Emilio Páez De La Mora | A01801224

22 de septiembre de 2026

Contenido

Introducción y trazabilidad de la rúbrica.....	3
1. Contexto del reto	4
1.1 Contexto y definición del problema.....	4
1.2 Disclaimer de selección e implementación de los modelos.....	5
2. Datos.....	5
2.1 Descripción del dataset	5
2.2 Limpieza de datos (manejo de faltantes).....	6
2.3 Transformación de datos.....	7
3. Modelo	9
3.1 División en entrenamiento, validación y pruebas, y comparación.....	9
3.2 Configuración del modelo.....	10
3.3 Entrenamiento	11
4. Evaluación.....	11
4.1 Selección de medidas de desempeño	11
4.2 Interpretación de resultados	12
4.3 Selección del modelo adecuado.....	13
5. Refinamiento de hiperparámetros	15
6. Solución: interfaz para utilizar el modelo.....	15
7. Conclusiones del equipo.....	16
7.1 ¿Qué aprendieron del comportamiento de los modelos sobre este dataset a lo largo de los tres avances?	16
.....	
7.2 ¿Qué cambiarían en el pipeline de preprocesamiento sabiendo lo que saben ahora?	17
7.3 ¿Qué esperan mejorar en la siguiente iteración?	17
7.4 Limitaciones del proyecto	17
7.5 Siguiendo pasos	18
8. Referencias y entregables relacionados.....	18

Introducción y trazabilidad de la rúbrica

Este documento consolida los hallazgos de los tres avances del equipo en el reto *Spaceship Titanic* (Kaggle), con el objetivo de documentar el proceso completo de tratamiento de datos, construcción, evaluación y refinamiento de un modelo de aprendizaje automático, así como su implementación en una interfaz utilizable. El código fuente completo se encuentra en el repositorio del equipo (Sección 8); este documento permite verificar cada indicador y cada sección del proyecto sin necesidad de revisar el código.

La siguiente tabla ubica cada indicador evaluado en la rúbrica del Reto en la sección de este documento donde queda evidenciado:

Tabla 1. Índice de cumplimiento de la rúbrica por sección

Rubro	Indicador de la rúbrica	Sección
Datos	Limpieza de datos con ETLs o herramienta equivalente	Sección 2.2
Datos	Transforma los datos con ETLs o herramienta equivalente	Sección 2.3
Modelo	Separa los datos en entrenamiento, validación y pruebas	Sección 3.1
Modelo	Configuración de Modelo	Sección 3.2
Modelo	Entrenamiento de Modelo	Sección 3.3
Evaluación	Selecciona medidas de desempeño adecuadas	Sección 4.1
Evaluación	Interpretación de los resultados del modelo	Sección 4.2
Evaluación	Selección de Modelo	Sección 4.3
Refinamiento	Ajuste de hiperparámetros	Sección 5
Solución	Interfaz para utilizar el modelo	Sección 6

1. Contexto del reto

1.1 Contexto y definición del problema

El equipo de rescate espacial enfrenta la imposibilidad de identificar a los pasajeros desaparecidos tras la colisión con una anomalía cósmica que corrompió los sistemas de registro, provocando retrasos críticos en las expediciones de búsqueda y un alto riesgo de abandono de víctimas. La decisión de “negocio” que este proyecto busca apoyar es ayudar al comandante de rescate a decidir qué lista exacta de pasajeros y áreas priorizar en la búsqueda, ya que actualmente los registros están mezclados y no distinguen entre pasajeros a salvo y transportados.

Se formuló como un problema de aprendizaje supervisado de clasificación binaria: predecir si un pasajero fue transportado (*Transported*: True/False) a partir de su estado (por ejemplo, *CryoSleep*), ubicación en la nave (cabina, cubierta), gastos en amenidades (*RoomService*, *Spa*, *VRDeck*, etc.) y características demográficas (edad, planeta de origen). El *baseline* de referencia es una heurística simple o un modelo que prediga siempre la clase mayoritaria; el éxito se definió como maximizar el *recall* de la clase *Transported* = True (para mitigar los falsos negativos, el error crítico: dejar de buscar a alguien que sí fue transportado) manteniendo al menos 80% de *accuracy* y un *F1-score* que supere el *baseline* heurístico.

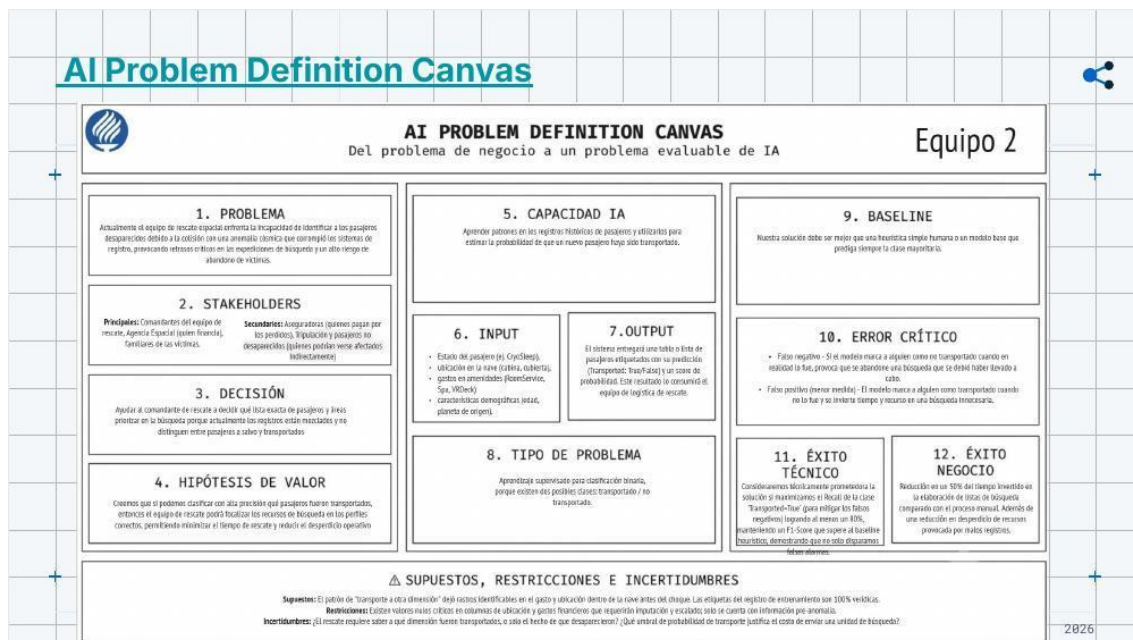


Figura 1. AI Problem Definition Canvas

En resumen, el reto consiste en predecir si un pasajero de la nave fue transportado a otra dimensión a partir de los datos demográficos, de reserva y de consumo a bordo, y es un problema de clasificación binaria.

El trabajo se desarrolló en tres avances documentados en *Notebooks/* del repositorio, y se consolidó en *Reto_Full/Spaceship_Titanic_Solucion.ipynb*, que es la referencia de este documento.

1.2 Disclaimer de selección e implementación de los modelos

Por falta de información sobre el contexto operativo de la misión de rescate, se asume que el costo de clasificar a un pasajero como *Transported* = False cuando en realidad sí fue transportado (falso negativo: abandonar una búsqueda que debía realizarse) supera con creces el costo del error contrario (falso positivo: destinar recursos a una búsqueda innecesaria). Esa asimetría, sin embargo, se asume de forma cualitativa, pues el proyecto no cuenta con una estimación del costo relativo entre ambos errores, y una métrica de optimización requiere una ponderación explícita.

Optimizar directamente el *recall* no es viable. Un modelo que prediga *Transported* = True para todos los pasajeros alcanza un *recall* de 1.0 y sería seleccionado por cualquier búsqueda de hiperparámetros guiada por esa métrica, pese a carecer de valor operativo (con clases balanceadas, su *accuracy* rondaría el 50%). Por eso se adoptó el F1-score como criterio de búsqueda durante el ajuste de hiperparámetros: al combinar *precision* y *recall* con el mismo peso, no obliga a comprometerse con una razón de costos que se desconoce y, al mismo tiempo, impide que el modelo degenere hacia la clase positiva.

La prioridad operativa del *recall* se conserva, pero se aplica en la etapa de selección y no en la de búsqueda: entre modelos que ya cumplen la restricción de $accuracy \geq 0.80$ y muestran un *F1-score* comparable, se elige el de mayor *recall*. Esa es exactamente la decisión documentada en la Sección 4.3, donde se adopta el SVC sobre el *Random Forest* por reducir los falsos negativos de 168 a 159 sin sacrificar especificidad.

2. Datos

2.1 Descripción del dataset

Los datos provienen de la competencia de *Spaceship Titanic* de Kaggle. *train.csv* contiene 8693 filas y 14 columnas (incluye la variable *Transported*); *test.csv* contiene 4277 filas y 13 columnas (sin la variable objetivo). Las clases del *target* están prácticamente balanceadas: 50.44%

Transported = True y 49.56% *Transported* = False, por lo que la métrica *accuracy* es confiable para la solución de este proyecto sin necesidad de realizar técnicas de balanceo de clases.

2.2 Limpieza de datos (manejo de faltantes)

Todas las variables del *dataset* presentaban valores faltantes. En vez de imputar todo de forma genérica (media/moda), se aprovechó una relación lógica descubierta en el EDA: los pasajeros en *CryoSleep* no pueden gastar en amenidades a bordo, por lo que un gasto total mayor a cero implica *CryoSleep* = False, y viceversa.

Esta imputación lógica se aplicó antes que cualquier imputación estadística:

Tabla 2. Procedimiento de limpieza aplicado a las variables relevantes

Variable	Acción de limpieza	Razón
<i>CryoSleep</i>	Imputación lógica: • Si gasto > 0 → <i>CryoSleep</i> = False • Si gasto = 0 → <i>CryoSleep</i> = True • Moda para el resto y binarización	Fuerte correlación con el costo total: quien está en <i>CryoSleep</i> no puede consumir amenidades.
<i>RoomService</i> , <i>FoodCourt</i> , <i>ShoppingMall</i> , <i>Spa</i> , <i>VRDeck</i>	Imputación lógica: • 0 cuando <i>CryoSleep</i> = True • Mediana para el resto	Mismo criterio lógico que para <i>CryoSleep</i> , evita imputar con la media en variables con sesgo alto.
<i>Age</i>	Mediana	Distribución razonablemente simétrica (<i>skewness</i> = 0.42); la mediana no distorsiona la forma.
<i>Cabin</i>	Separada en <i>Deck</i> , <i>CabinNum</i> y <i>Side</i> (split por “/”)	Cardinalidad inmanejable como texto único; sus tres componentes tienen significado propio.

<i>CabinNum</i>	Mediana y agrupación en <i>bins</i> de 300 (<i>CabinNumBin</i>)	Captura la sección física de la nave sin usar más de 1000 valores únicos como categoría.
<i>HomePlanet, Destination, Deck, Side</i>	Moda para nulos	Variables categóricas sin relación obvia con otra columna para imputación lógica.
<i>VIP, Name</i>	Descartadas	<i>VIP</i> con >95% de valores en False (sin poder discriminante). <i>Name</i> no tiene valor predictivo.

Todas las estadísticas de imputación (medianas, modas) se calculan únicamente sobre los datos de entrenamiento y se aplican después a validación y prueba, para evitar fuga de información (*data leakage*). Esta lógica está encapsulada en la clase *SpaceshipPreprocessor* (métodos *fit* y *transform*, ver *preprocessing.py* o el bloque de código comentado), reutilizable como *pipeline* de *scikit-learn*.

2.3 Transformación de datos

Tras la limpieza, se aplicaron las siguientes transformaciones para dejar el *dataset* listo para el modelado:

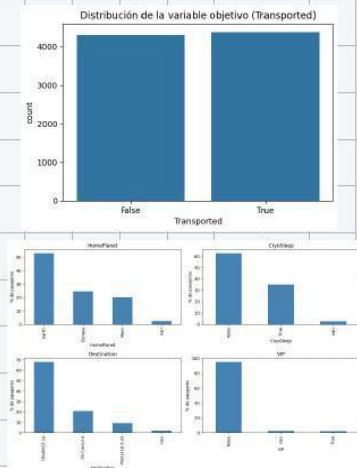
Tabla 3. Justificación y procedimiento de transformación de datos

Variable(s)	Transformación	Justificación
<i>RoomService, FoodCourt, ShoppingMall, Spa, VRDeck</i>	$\log_{1p}(x) \rightarrow \text{RobustScaler}$	Sesgo severo (<i>skewness</i> entre 6 y 9) y <i>outliers</i> extremos; <i>log1p</i> comprime la cola larga y <i>RobustScaler</i> usa mediana/IQR en vez de media/desviación estándar.

<i>Age, group_size</i>	<i>RobustScaler</i>	Mismo escalador que los gastos, por consistencia y robustez a <i>outliers</i> .
<i>HomePlanet, Destination, Deck, CabinNumBin</i>	<i>One-Hot encoding (drop_first = True)</i>	Catóricas nominales sin jerarquía natural.
<i>CryoSleep, Side</i>	Codificación binaria (0/1)	Variables ya binarias por naturaleza.
<i>PassengerId</i>	Extraído a <i>group_size</i> (tamaño del grupo de viaje) y eliminado	El identificador en sí no aporta información, pero el tamaño del grupo sí se asocia al <i>target</i> (ver el Apéndice del <i>notebook</i> de consolidación).
<i>HasSpent</i>	Variable derivada: 1 si la suma de gastos > 0	Señal binaria redundante pero útil para el modelo, correlacionada con <i>CryoSleep</i> .

El resultado final es un *dataset* de 8693 filas y 27 columnas, sin valores nulos, listo para el entrenamiento. Todas las transformaciones numéricas (*log1p(x)*, *RobustScaler*) se ajustan solo con datos de entrenamiento y se aplican después sobre validación, prueba y datos nuevos. El mismo objeto *SpaceshipPreprocessor* se reutiliza en la interfaz web (Sección 6).

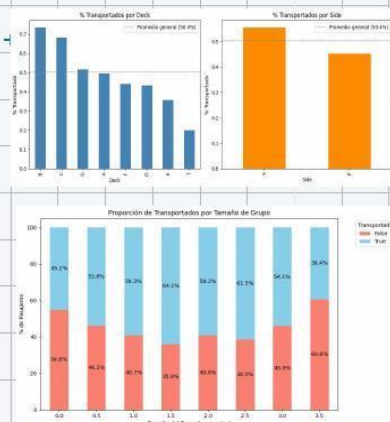
Análisis Exploratorio de Datos (EDA).



- **Clases balanceadas:** 50.4% transportados / 49.6% no transportados. Al no haber una clase dominante, el accuracy es una métrica confiable para evaluar el modelo.
- **CryoSleep** es el feature clave: los pasajeros dormidos (en animación suspendida) tienen una tasa de transporte mucho más alta que los despiertos.
- **Descubrimiento útil:** estos mismos pasajeros dormidos no gastaron nada a bordo, esto nos permitió tratar datos faltantes con lógica, en vez de solo con promedios.

Figura 2. Distribución del target y variables categóricas clave del EDA

Análisis Exploratorio de Datos (EDA).



- **La ubicación en el barco es importante:** los decks B y C tienen ~68-73% de probabilidad de transporte, mientras que E y F caen a ~36-44%, casi el doble de diferencia entre secciones, muy por encima del promedio general (50.4%)
 - Se aprecia que A, B, y C se encuentran cercanos o por encima del promedio general (excluyendo a G), lo que puede indicar ordinalidad en deck.
 - Viajar en estribor (Side = S) da más probabilidad de ser transportado, que estar en babor (Side = P)
- **Viajar acompañado también influyó:** el tamaño del grupo de viaje se relaciona con la probabilidad de ser transportado.
- **Descubrimiento útil:** el tamaño del grupo de viaje se relaciona con la probabilidad de ser transportado, así que lo convertimos en una variable nueva, llamada "group_size".

Figura 3. Relación entre Deck, Side, tamaño de grupo y probabilidad de transporte

3. Modelo

3.1 División en entrenamiento, validación y pruebas, y comparación

Los datos preprocesados se dividieron con `train_test_split` (80% entrenamiento / 20% prueba, estratificado por la variable objetivo, `random_state = 42`), de forma que el conjunto de prueba se mantiene completamente aislado y solo se usa para la evaluación final de cada modelo. Dentro del

conjunto de entrenamiento, la validación se realizó con validación cruzada *K-Fold* estratificada ($K = 5$): tanto las curvas de aprendizaje y validación (Sección 4.2) como la búsqueda de hiperparámetros (Sección 5) usan ese esquema, que da una estimación más estable del desempeño en datos no vistos que un único *split* adicional, sin sacrificar datos de entrenamiento.

Como punto de partida, se calculó un *baseline* (predecir siempre la clase mayoritaria) y se compararon dos modelos de familias distintas: regresión logística (lineal) y *Random Forest* (ensamble de árboles), evaluados sobre el mismo conjunto de prueba:

Tabla 4. Comparación del *baseline*, la regresión logística y el *Random Forest*

Modelo	Accuracy	Precision	Recall	F1-score
<i>Baseline</i>	0.5036	-	-	-
Regresión logística	0.7936	0.7869	0.8090	0.7980
<i>Random Forest</i> ($n_estimators = 200$, $max_depth = 10$)	0.7987	0.8009	0.7991	0.8000

El *Random Forest* superó a la regresión logística en *accuracy* (+0.0051), *precision* (+0.0140) y *F1-score* (+0.0020), pero perdió ligeramente en *recall* (−0.0099). La diferencia es pequeña, pero se eligió *Random Forest* como modelo del equipo porque fue el único con mejora consistente en la métrica compuesta (*F1-score*) y porque, al ser un ensamble de árboles, tiene más margen de mejora vía ajuste de hiperparámetros que un modelo lineal.

3.2 Configuración del modelo

La configuración del *Random Forest* del equipo fue $n_estimators = 200$ y $max_depth = 10$, elegida manualmente en el segundo avance. Antes de ajustarla de forma automática, se diagnosticó con curvas de aprendizaje y de validación (Sección 4.2) para entender si el modelo estaba en

underfitting, *overfitting* o cerca de su punto óptimo, y así centrar la búsqueda de hiperparámetros en el rango correcto en vez de explorar a ciegas.

- Curva de aprendizaje: *overfitting* moderado y decreciente. El score de entrenamiento baja de 0.974 a 0.871 conforme crece el *dataset*, mientras la validación sube de 0.778 a 0.807. La brecha se reduce, pero no cierra del todo.
- Curva de validación (*max_depth*): pico claro de generalización en *max_depth* = 10 (*F1-score* = 0.806). Antes de ese punto hay *underfitting*; después hay *overfitting* creciente sin beneficio en validación.
- Curva de validación (*min_samples_leaf*): el *F1-score* de validación se mantiene casi plano entre 1 y 5 (entre 0.806 y 0.807), y después baja. La brecha entre entrenamiento y validación se cierra porque el entrenamiento empeora, no porque la validación mejore.

Con ese diagnóstico, el rango de búsqueda para el *tuning* (Sección 5) se centró en *max_depth* entre 8 y 14 y *min_samples_leaf* entre 1 y 5, en vez de explorar el espacio completo de hiperparámetros a ciegas.

3.3 Entrenamiento

El modelo se entrena con *RandomForestClassifier* (*random_state* = 42) sobre *X_train* y *y_train* (conjunto de entrenamiento del *split* 80/20), usando el *dataset* ya preprocesado (27 columnas, sin nulos, escaladas). El mismo esquema de entrenamiento (*fit* sobre entrenamiento, *predict* y evaluación sobre prueba) se repite para cada configuración de hiperparámetros probada en la validación cruzada y en el ajuste fino (Sección 5), y para los cinco modelos individuales del componente por integrante (Sección 4.3).

4. Evaluación

4.1 Selección de medidas de desempeño

Con las clases prácticamente balanceadas (50.44% / 49.56%), *accuracy* es una métrica confiable como referencia general. Se complementa con *precision*, *recall* y *F1-score*, ya que, aunque *accuracy* resume el desempeño global, no distingue qué tipo de error comete el modelo. Considerando nuestro *AI Problem Definition Canvas* (Sección 1), el error crítico del problema de negocio es el falso negativo: marcar a alguien como no transportado cuando en realidad sí lo fue provoca que se abandone una búsqueda que se debió haber llevado a cabo; el falso positivo (invertir recursos en una búsqueda innecesaria) es un error de menor costo. Por eso *recall* es la métrica

priorizada al final del modelo, mientras que *F1-score* se usó como criterio principal durante el ajuste de hiperparámetros, y ROC-AUC se añadió como segunda métrica porque evalúa la calidad del ordenamiento de probabilidades en todos los umbrales posibles, no solo en el de decisión 0.5, lo que ayuda a confirmar si una mejora es robusta o depende de un ajuste específico del umbral.

4.2 Interpretación de resultados

Se compararon tres versiones del *Random Forest* para aislar el efecto de cada paso: sin ajustar, configurada a mano en el Avance 2 y ajustada con *RandomizedSearchCV* en el Avance 3. Las tres superan por mucho el *baseline* de la clase mayoritaria (*accuracy* = 0.5036 en la Tabla 4), del que solo se reporta *accuracy* porque predice una sola clase:

Tabla 5. Resultados de las tres versiones del *Random Forest*

Versión	Accuracy	F1-score	ROC-AUC
RF sin ajustes	0.7936	0.8003	0.8028
RF configurado a mano	0.7987	0.8000	0.8972
RF ajustado con <i>RandomizedSearchCV</i>	0.8028	0.8050	0.8984

El modelo ajustado acertó 1396 de 1739 casos del conjunto de prueba (matriz de confusión: 688 verdaderos negativos, 175 falsos positivos, 168 falsos negativos, 708 verdaderos positivos). Los dos tipos de error están casi parejos (175 vs. 168), es decir, el modelo no se equivoca sistemáticamente hacia un lado. La mejora del *F1-score* en el conjunto de prueba (0.8050) sobre el *Random Forest* configurado a mano es real pero modesta, consistente con el diagnóstico de la Sección 3.2: el modelo base ya estaba cerca de un buen equilibrio sesgo-varianza, por lo que seguir ajustando ese mismo modelo ofrece retornos decrecientes.

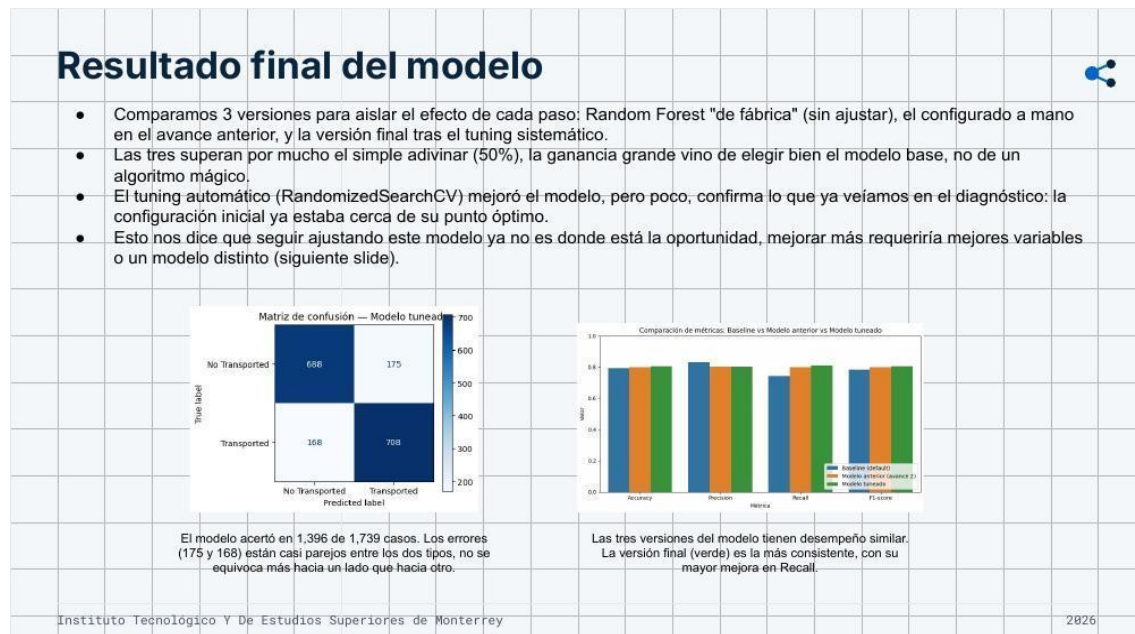


Figura 4. Matriz de confusión y comparación de métricas de los Random Forest

4.3 Selección del modelo adecuado

Cada integrante del equipo ajustó de forma independiente un modelo de una familia distinta a la del *Random Forest* y distinta entre sí, evaluado con las mismas métricas del equipo sobre el mismo conjunto de prueba:

Integrante	Modelo	F1-score	ROC-AUC	¿Superó al RF del equipo?	¿Cambiaría la decisión?
Ana Paula Moreno	Regresión Logística	0.7934	0.8677	No	No
David Tinoco Romero	SVC	0.8106	0.8945	Sí	Sí
Emilio Páez de la Mora	KNN	0.7603	0.8747	No	No
Emilio Torres Paul	Naive Bayes	0.7068	0.7876	No	No
Alejandro Vázquez	Red Neuronal (MLP)	0.7917	0.8881	No	No

Figura 5. Métricas de evaluación de los modelos de distintas familias desarrollados por cada integrante

- Regresión logística (Ana Paula): por debajo del RF en las métricas compuestas. El problema tiene relaciones no lineales que un modelo lineal no captura tan bien.
- SVC (David): el único modelo individual que superó al *Random Forest* del equipo en *accuracy*, *precision*, *recall* y *F1-score*.

- KNN (Emilio Páez): por debajo del RF, sobre todo en *recall* (0.7249 vs. 0.8082); con 27 dimensiones (varias *dummies* de baja cardinalidad), la maldición de la dimensionalidad diluye la noción de cercanía que usa KNN para votar.
- *Naive Bayes* (Emilio Torres): por debajo del RF; el supuesto de independencia condicional entre variables es poco realista aquí (*CryoSleep* y las variables de gasto están fuertemente correlacionadas por construcción).
- Red neuronal / MLP (Alejandro): por debajo del RF; con un *dataset* de tamaño moderado (alrededor de 7000 filas de entrenamiento), una red neuronal no tiene suficiente volumen de datos para superar a un ensamble de árboles ya bien ajustado.

Por ese resultado, se reconstruyó el SVC con sus hiperparámetros óptimos ($kernel = rbf$, $C = 150$, $gamma = 0.005$) y se comparó directamente contra el *Random Forest* ajustado, ambos sobre el mismo conjunto de prueba:

Métrica	Random Forest tuneado	SVC tuneado	Diferencia (SVC – RF)
Accuracy	0.8028	0.8074	+0.0046
<u>Precision</u>	0.8018	0.8029	+0.0011
Recall	0.8082	0.8185	+0.0103
F1-score	0.8050	0.8106	+0.0056
ROC-AUC	0.8984	0.8945	-0.0039

Figura 6. Comparación final entre el *Random Forest* y el SVC

El SVC captura más verdaderos positivos (717 vs. 708) y reduce falsos negativos (159 vs. 168) frente al RF, sin sacrificar especificidad (687 vs. 688 verdaderos negativos), lo que se traduce en un *recall* notablemente mejor y, con ello, en mejores *F1-score* y *accuracy*. El RF mantiene una ligera ventaja en ROC-AUC (0.8984 vs. 0.8945): su ordenamiento de probabilidades a lo largo de todos los umbrales es marginalmente mejor, aunque en el punto de operación real (umbral 0.5) el SVC gana. Dado que *recall* es la métrica priorizada por el problema de negocio, adoptamos una estrategia dual: SVC como implementación principal (mejor *recall*, con la infraestructura computacional adecuada) y *Random Forest* como protocolo de respaldo para operación manual o interpretación, ya que sus reglas de división explícitas permiten justificar una predicción sin depender de una API o servidor corriendo. Por eso la interfaz de la Sección 6 permite elegir entre ambos modelos.

5. Refinamiento de hiperparámetros

El refinamiento de hiperparámetros del modelo del equipo (*Random Forest*) se hizo en dos niveles: una búsqueda sistemática a nivel de equipo y una exploración individual de un hiperparámetro distinto por integrante en el Avance 2 (*min_samples_leaf*, *max_depth*, *min_samples_split*, *n_estimators*), que confirmó que ningún cambio aislado producía una mejora clara, lo cual motivó el diagnóstico más riguroso con curvas de aprendizaje y validación descrito en la Sección 3.2.

Con el rango ya acotado por ese diagnóstico (*max_depth* entre 8 y 14, *min_samples_leaf* entre 1 y 5), el espacio de búsqueda que combina los cinco hiperparámetros (*n_estimators*, *max_depth*, *min_samples_leaf*, *min_samples_split*, *max_features*) tiene 2100 combinaciones posibles. Con *cv* = 5, un *GridSearchCV* exhaustivo hubiera implicado 10 500 entrenamientos de *Random Forest*, costoso para la ganancia marginal que ya anticipaban las curvas de validación. Se usó en su lugar *RandomizedSearchCV* con *n_iter* = 20 (20 combinaciones elegidas al azar dentro del espacio definido) y validación cruzada *StratifiedKFold* de *K* = 5, con *F1-score* como criterio principal de búsqueda y ROC-AUC como segunda métrica de verificación.

Los mejores hiperparámetros encontrados fueron *max_depth* = 12, *min_samples_leaf* = 5, *min_samples_split* = 5, *n_estimators* = 300 y *max_features* = *sqrt*, con *F1-score* en validación cruzada = 0.8100 y ROC-AUC en validación cruzada = 0.8888. Esta configuración confirma el diagnóstico: una profundidad ligeramente mayor a 10, pero compensada con hojas y *splits* más exigentes (más regularización), no un árbol simplemente más profundo sin control.

Además del ajuste del modelo de equipo, cada integrante ajustó al menos tres hiperparámetros de su modelo individual con búsqueda en cuadrícula o aleatoria y validación cruzada (por ejemplo: David: *C*, *kernel* y *gamma* del SVC con *RandomizedSearchCV* seguido de un *GridSearchCV* refinado; Emilio Páez: *n_neighbors*, *weights* y *metric* del KNN con *GridSearchCV*; Alejandro: *hidden_layer_sizes*, *alpha*, *learning_rate_init* y *activation* del MLP con *RandomizedSearchCV*; Emilio Torres: *n_components*, *whiten* y *var_smoothing* de un *pipeline* PCA + *Naive Bayes* con *GridSearchCV*), documentados con su matriz de confusión y curva ROC en la Sección 4 del Avance 3.

6. Solución: interfaz para utilizar el modelo

Se construyó una aplicación web (*Flask*) que expone los dos modelos de la estrategia dual (*Random Forest* y SVC ajustado) junto con el preprocesador ya ajustado (*SpaceshipPreprocessor*,

exportado con *joblib* desde el mismo *notebook* de consolidación), de forma que la interfaz predice sin necesidad de reentrenar nada.

- Predicción individual: un formulario (*index.html*) captura los datos de un pasajero (*HomePlanet*, *CryoSleep*, *Destination*, *Age*, cabina, gastos por categoría) y los envía a la ruta POST */predict*; el *backend* arma un *DataFrame* de una fila, lo transforma con el preprocesador y devuelve la predicción (*Transported*: True/False) junto con la probabilidad, mostrada con una barra visual.
- Predicción por lote: una segunda sección de la misma interfaz permite subir un archivo .csv o .xlsx con varios pasajeros a la ruta POST */predict_batch*; el *backend* infiere *group_size* si no viene *PassengerId*, aplica el mismo preprocesador y devuelve una tabla con las predicciones, estadísticas resumen (total, transportados, probabilidad promedio) y un botón para descargar los resultados en .csv.
- En ambos flujos, el usuario puede elegir qué modelo de la estrategia dual usar (SVC o *Random Forest*) desde un selector, lo que materializa en la interfaz la decisión de la Sección 4.3.

El archivo *pasajeros_prueba.xlsx* incluido en el repositorio sirve como ejemplo de entrada para probar el flujo de predicción por lote de la interfaz.

7. Conclusiones del equipo

7.1 ¿Qué aprendieron del comportamiento de los modelos sobre este dataset a lo largo de los tres avances?

Aprendimos que el rendimiento no siempre crece de manera proporcional a la complejidad computacional. Los modelos basados en ensambles de árboles (*Random Forest*) alcanzaron muy rápido un techo de rendimiento con configuraciones relativamente conservadoras (*max_depth* = 10), y el ajuste exhaustivo posterior ofreció retornos decrecientes, mejorando el *F1-score* de manera casi marginal (alrededor de +0.01 en validación cruzada). Por otro lado, comprobamos que los modelos de frontera geométrica (SVC con *kernel rbf*) tienen la capacidad de proyectar las interacciones de este *dataset* a espacios donde logran aislar mejor la clase positiva, consiguiendo un salto real en *recall*. Finalmente, aprendimos a valorar el *trade-off* operativo: un SVC puede ganar en precisión estricta, pero un *Random Forest* gana en interpretabilidad y estabilidad. En

ciencia de datos, el “mejor modelo” depende de las condiciones de implementación, de ahí la estrategia dual de la Sección 4.3.

7.2 ¿Qué cambiarían en el pipeline de preprocesamiento sabiendo lo que saben ahora?

Sabiendo que un modelo sensible a las distancias geométricas (SVC) terminó ofreciendo el mejor *recall*, el preprocesamiento debió enfocarse más en la distribución de las variables numéricas. En vez de un escalado estándar, se podrían explorar transformaciones como *Yeo-Johnson* para suavizar el sesgo en las variables de gasto de forma más flexible que $\log_{1p}(x)$. También repensaríamos el tratamiento de *Deck* y *CabinNum*: una exploración adicional del equipo mostró que tratar *Deck* como variable ordinal (ordenada empíricamente por su tasa de *Transported*, en vez de *One-Hot encoding*) da un *F1-score* ligeramente mejor (0.8014 vs. 0.8000) y reduce el *dataset* de 27 a 21 columnas, aunque con la advertencia de que ese orden usa información del *target* y por eso no reemplazó la codificación *One-Hot* del *pipeline* principal.

7.3 ¿Qué esperan mejorar en la siguiente iteración?

Para una futura iteración, el objetivo sería enfocarnos agresivamente en *feature engineering* adicional (por ejemplo, proporción de gasto por categoría dentro de cada pasajero) y experimentar con arquitecturas de *Gradient Boosting* (*XGBoost*, *LightGBM*), que combinan la robustez de los árboles con optimización secuencial. Un *Stacking Ensemble* que combine el ordenamiento probabilístico del *Random Forest* con la sensibilidad de frontera del SVC es también un camino natural, dado que ambos modelos demostraron fortalezas complementarias.

7.4 Limitaciones del proyecto

- Valores nulos en columnas críticas de ubicación y gastos financieros: se resolvieron con imputación lógica y estadística, pero se asume que las etiquetas de entrenamiento son 100% verídicas y que el patrón de “transporte a otra dimensión” dejó rastros identificables en el gasto y la ubicación antes del choque. Son supuestos razonables, pero no verificables con los datos disponibles.
- El orden ordinal explorado para *Deck* (Apéndice del *notebook* de consolidación) se derivó de la tasa de *Transported* calculada sobre *train*, es decir, usa información del *target*. Es válido como exploración (*target encoding*), pero no debe confundirse con una propiedad física de la nave, por lo que no reemplaza la codificación *One-Hot* del *pipeline* principal.

- Con 27 dimensiones tras el *One-Hot encoding*, los modelos sensibles a la distancia (KNN) se ven perjudicados por la maldición de la dimensionalidad, y el *dataset* (alrededor de 7000 filas de entrenamiento) es de tamaño moderado para una red neuronal, que no logró superar al *Random Forest*.
- El SVC final, aunque es el mejor modelo en *recall*, *accuracy*, *precision* y *F1-score*, es una “caja negra” matemática difícil de interpretar sin cómputo, a diferencia del *Random Forest*. De ahí que la solución mantenga ambos modelos disponibles en vez de descartar uno.

7.5 Siguientes pasos

- Recolectar o simular un volumen mayor de datos de mejor calidad, en particular para reducir la incertidumbre sobre grupos de viaje grandes (pocas observaciones) y sobre las combinaciones menos frecuentes de *Deck* y *Destination*.
- Probar transformaciones más flexibles (*Yeo-Johnson*) para las variables de gasto y evaluar si mejoran el desempeño del SVC, que es sensible a la escala y forma de las variables numéricas.
- Explorar arquitecturas de *Gradient Boosting* (*XGBoost*, *LightGBM*) y un *Stacking Ensemble* de RF y SVC como trabajo futuro, aprovechando que ambos modelos mostraron fortalezas complementarias (*recall* frente a ROC-AUC).

8. Referencias y entregables relacionados

- Repositorio público con el código fuente completo (*notebooks* de los tres avances, *notebook* de consolidación, *preprocessing.py*, *app.py*, *index.html*) e instrucciones de ejecución (*README*).
- <https://github.com/A01801224/TC3009C-RI/tree/main>